漫步数码时代，

铸就精彩人生，

程序员修炼之路，

从精雕算法开始……

第2章 分治策略

(Divide and Conquer)

(一)

主讲人：周正深

上一章重要内容回顾

主定理：设$a\geq 1$，$b>1$为常数，$f(n)$为函数，$T(n)$为非负整数，且

$$T(n)=aT(n/b)+f(n)$$

则有以下结果：

- 若$f(n)=O(n^{\log_b a+\varepsilon}),\varepsilon<0$，那么$T(n)=\Theta(n^{\log_b a})$
- 若$f(n)=\Theta(n^{\log_b a+\varepsilon}),\varepsilon=0$，那么$T(n)=\Theta(n^{\log_b a}\log n)$
- 若$f(n)=\Omega(n^{\log_b a+\varepsilon}),\varepsilon>0$，且对某个常数 $c<1$和所有充分大的 n 有 $a f(n/b)\leq c f(n)$，那么

$$T(n)=\Theta(f(n))$$

2.1 分治策略的基本思想

2.1.1 两个熟悉的例子

2.1.2 分治算法的一般性描述

2.2 分治算法的分析技术

2.3 改进分治算法的途径

2.3.1 通过代数变换减少子问题个数

2.3.2 利用预处理减少递归内部的计算量

2.4 典型实例

2.4.1 快速排序算法

2.4.2 选择问题

2.4.3 $n-1$次多项式在全体$2n$次方根上的求值

2.1 分治策略的基本思想

对于一个规模为n的问题：

1. 若该问题可以容易地解决（比如说规模n较小）则直接解决；

2. 否则将其分解为k个规模较小的子问题，这些子问题互相独立且与原问题形式相同；

3. 递归地解这些子问题；

4. 然后将各子问题的解合并得到原问题的解。

这种算法设计策略叫做分治策略。

例2.1 芯片测试问题

已知：有n片芯片，好芯片至少比坏芯片多1片。

假设：好芯片测试结果是正确的，坏芯片测试结果是不确定的。

问题：使用最少测试次数，从中挑出1片好芯片。

用A芯片对B芯片进行测试，结果分析如下：

例2.1 芯片测试问题

用A芯片对B芯片进行测试，报告如下：

A 芯片	B 芯片	结论
好的	好的	B 是好的
好的	坏的	B 是坏的
坏的	好的	B是好的或B是坏的
坏的	坏的	B是好的或B是坏的

例2.1 芯片测试——一般算法

芯片测试的一般算法思想：

若芯片数<=2，则都是好芯片，算法结束。

任取一个芯片，用剩余n-1的芯片检测它，若(n-1)的半数及以上认为它是好芯片，则为好芯片，算法结束。

否则，该芯片为坏芯片，去掉此芯片，对剩余n-1芯片重复以上过程。

例2.1 芯片测试——一般算法

算法正确性证明：对于被测芯片x，若n-1个芯片半数及以上认为它是好的，而此芯片必定是好芯片。

反证法：对于被测芯片x，若n-1个芯片半数及以上认为它是好的，而此芯片是坏芯片。

设剩下的n-1个芯片中好芯片h个，坏芯片k个。

$$h + k = n - 1$$

因为x是坏的，所以 $h >= k + 2$

由于好芯片h比坏芯片至少多2个，所以应该检测x是坏芯片的报告比好芯片至少多两个。那么x是好芯片的报告绝无可能得到半数及以上。这与命题相矛盾，则x芯片不是坏芯片。

证明完毕。

例2.1 芯片测试——一般算法

芯片测试的一般算法思想：

若芯片数<=2，则都是好芯片，算法结束。

任取一个芯片，用剩余n-1的芯片检测它，若(n-1)的半数以上认为它是好芯片，则为好芯片，算法结束。

去掉后此芯片，对剩下芯片重复以上过程。

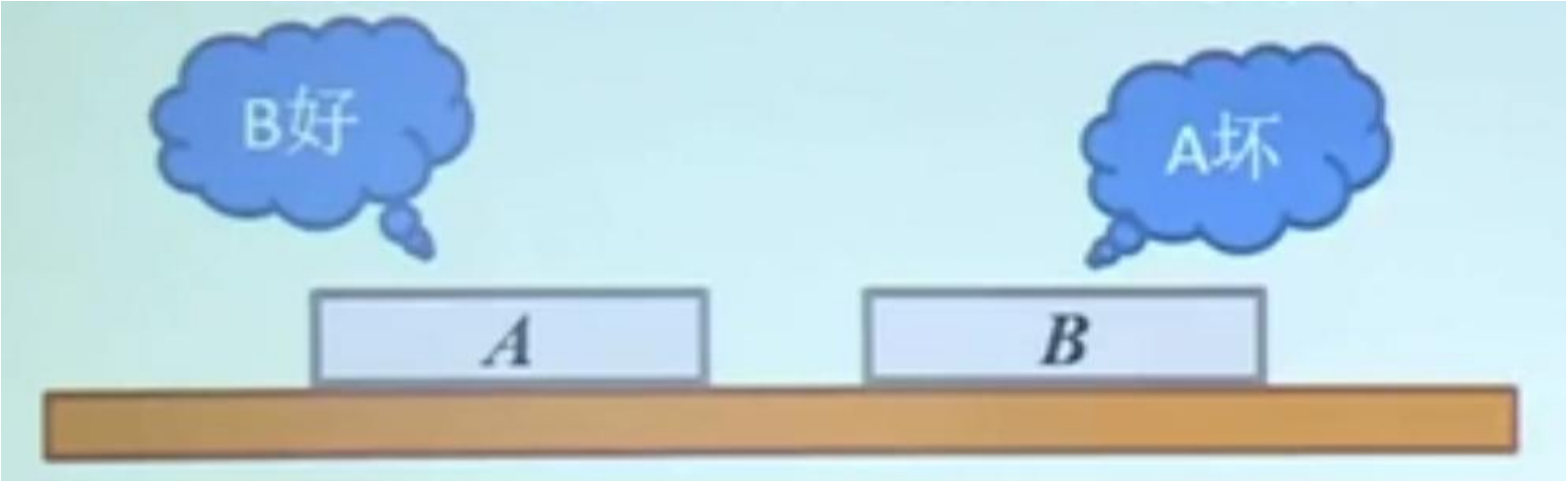
递推方程：$\begin{cases} W(n) = W(n-1) + n - 1 \\ W(2) = 0 \\ W(1) = 0 \end{cases}$

算法复杂度： $T(n)=O(n^2)$

例2.1 芯片测试—分治算法

分治算法：

放在工作台上的两片芯片同时互测，测试结果如下：



结论	A 报告	B 报告	结论
①	B是好的	A是好的	A,B 都好或 A,B 都坏
②	B是好的	A是坏的	至少一片是坏的
③	B是坏的	A是好的	至少一片是坏的
④	B是坏的	A是坏的	至少一片是坏的

例2.1 芯片测试—分治算法

分治算法思想：

①若n<=3，算法可直接解得，算法结束。

②否则，若n为奇数，分别对其中某个芯片x进行测试，判断出x是好芯片，还是坏芯片。好芯片算法结束，坏芯片丢掉继续下一步。

③此时芯片为偶数，两两一组测试，按以下规则淘汰后芯片进入下一轮。如果测试结果是情况1，那么A、B中留1片，丢1片；如果是后三种情况，则把A和B全部丢掉。

④重复以上过程(从①开始下一轮)。

例2.1 芯片测试——分治算法的伪码描述

算法2.3 Test(n)

1. $k \leftarrow n$
2. while $k > 3$ do
3. 　　将芯片分成 $\lfloor k/2 \rfloor$ 组 // 如有轮空芯片，特殊处理
4. 　　for $i = 1$ to $\lfloor k/2 \rfloor$ do
　　if 2片好，则任取1片留下
　　else 2片同时丢掉
5. 　　$k \leftarrow$剩下的芯片数
6. if $k == 3$ then
　　任取2片芯片测试
　　if 2好 or 1好1坏 then 取被测的芯片
　　else 任取1片检测芯片
7. if $k == 2$ or 1 then 任取1片

芯片检测问题分治算法复杂度

$$\begin{cases} W(n) = W\left(\frac{n}{2}\right) + O(n) & n > 3 \\ W(n) = 1 & n \le 3 \end{cases}$$

符合主定理第3种情况

$$\Rightarrow W(n) = O(n)$$

证明：

$$\because a = 1,\ b = 2,\ f(n) = O(n)$$

$$n^{\log_b a} = n^{\log_2 1} = n^0$$

$$f(n) = \Omega(n^{\log_b a + \varepsilon}) = n^{0+\varepsilon} = O(n)$$

$\therefore \varepsilon = 1 > 0$，符合第 3 种情况

$$\therefore W(n) = \Theta(f(n)) = \Theta(n)$$

算法2.3 Test(n)分治算法的证明

A 报告	*B 报告*	*结论*
B是好的	*A是好的*	*A,B 都好或 A,B 都坏*
B是好的	*A是坏的*	*至少一片是坏的*
B是坏的	*A是好的*	*至少一片是坏的*
B是坏的	*A是坏的*	*至少一片是坏的*

证明：

①若n<=2，根据题意，全为好的，算法成立。

②否则，若n为奇数，则检测轮空芯片，若为好的算法结束若为坏的，芯片去掉，则剩余芯片为偶数，且好芯片比坏芯片多2片。

例2.1 芯片测试—分治算法的证明

③对于偶数情况证明如下：

由原始条件或②得知，好芯片比坏芯片多2片

设测试结果中第一~四种情况分别为$k1$、$k2$、$k3$、$k4$个。

算法将$k2$、$k3$、$k4$中的芯片全部去掉， $k2$、$k3$、$k4$中至少有一半坏芯片，所以剩下的$k1$中好芯片比坏芯片至少多2片。

设有$k1$ 中有h对好的，k对坏的，则

$$2h-2k>=2$$

$$h-k>=1$$

即：在下一轮测试中芯片个数$h+k$片，好芯片比坏芯片至少多一片，规约到原命题。

算法证明完毕

例2.2 幂乘计算问题

问题：设a是给定实数，计算 a^n，n为自然数

传统算法：$a^n = a \times a \times .. a \times a$

传统算法复杂度：$W(n) = \Theta(n)$

例2.2 幂乘计算问题

分治法基本思想：

n为偶数：$\underbrace{a\ldots\ldots\ldots\ldots a}_{n/2\text{个}} \mid \underbrace{a\ldots\ldots\ldots\ldots a}_{n/2\text{个}}$

n为奇数：$\underbrace{a\ldots\ldots\ldots\ldots a}_{(n-1)/2\text{个}} \mid \underbrace{a\ldots\ldots\ldots\ldots a}_{(n-1)/2\text{个}} \mid a$

$$a^n = \begin{cases} a^{n/2} \times a^{n/2} & n \text{ 为偶数} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a & n \text{ 为奇数} \end{cases}$$

分治法复杂度：

$W(n) = W(n/2) + \Theta(1)$ $\qquad$ $W(n) = \Theta(\log n)$.

计算 Fibonacci 数

Fibonacci 数列

1, 1, 2, 3, 5, 8, 13, 21, …

满足 $F_n=F_{n-1}+F_{n-2}$ ，求F_n

增加$F_0=0$，得到数列 0, 1, 1, 2, 3, 5, 8, 13, 21, …

通常算法：从 $F_0, F_1, \ldots,$ 根据定义陆续相加，得到F_n

普通算法复杂度：$O(n)$

计算 Fibonacci 数——幂乘算法证明

定理2.1 设 $\{F_n\}$为 Fibonacci 数构成的数列，那么

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

归纳证明：当n=1时，

$$\text{左边} = \begin{bmatrix} F_2 & F_1 \\ F_1 & F_0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \text{右边}$$

等式成立

补充：矩阵相乘公式

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

$$c_{11} = a_{11} \times b_{11} + a_{12} \times b_{21}$$

$$c_{12} = a_{11} \times b_{12} + a_{12} \times b_{22}$$

$$c_{21} = a_{21} \times b_{11} + a_{22} \times b_{21}$$

$$c_{22} = a_{21} \times b_{12} + a_{22} \times b_{22}$$

计算 Fibonacci 数——幂乘算法证明

假设对于任意正整数n，命题成立，即

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

那么，当$n=n+1$时，代入左边得，（$F(n+2)=F(n+1)+F(n)$，$F(n+1)=F(n)+(F(n-1)$，$F(n)=F(n)$）

$$\text{左边} = \begin{bmatrix} F_{n+2} & F_{n+1} \\ F_{n+1} & F_n \end{bmatrix}$$

$$\text{右边} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n+1} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} F_{n+1}+F_n & F_{n+1} \\ F_n+F_{n-1} & F_n \end{bmatrix} = \begin{bmatrix} F_{n+2} & F_{n+1} \\ F_{n+1} & F_n \end{bmatrix} = \text{左边}$$

证毕。

计算 Fibonacci 数——幂乘算法

Fibonacci数列分治算法思想：

利用公式：$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$

令矩阵 $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$，用幂乘(分治)算法计算 M^n

Fibonacci数列分治算法复杂度：

矩阵乘法次数 $T(n) = \Theta(\log n)$

每次矩阵乘法需要8次元素相乘

总计元素相乘次数为 $\Theta(\log n)$

2.3 改进分治算法的途径

$$T(n)=aT(n/b)+f(n)$$

当a较大，b较小，$f(n)$不大时，方程的解：

$$T(n)=\Theta(n^{\log_b a})$$

通过减少问题的个数a、加大b即降低子问题的规模、把子问题内的工作量移到子问题外、降低$f(n)$的界，从而降低算法复杂的。

1. 通过代数变换，减少子问题个数

2. 利用预处理，减少递归内部操作

例3：位乘问题

2.3.1 通过代数变换，减少子问题个数

例2. 3 位乘问题：

设X,Y是 n 位二进制数， $n = 2^k$, 求 XY.

①蛮力算法：用Y的每一位去乘X的每一位，再求和.

算法复杂度：$O(n^2)$

例3：位乘问题

②一般分治法：

X	A	B
Y	C	D

令

$$X = A2^{n/2} + B, \quad Y = C2^{n/2} + D.$$

则

$$XY = AC\,2^{n} + (AD + BC)\,2^{n/2} + BD$$

$$\begin{cases} W(n) = 4W(n/2) + cn \\ W(1) = 1 \end{cases}$$

例3：位乘问题

$$\begin{cases} W(n) = 4W(n/2) + cn \\ W(1) = 1 \end{cases}$$

代入主定理

$$T(n) = aT(n/b) + f(n),\quad a = 4,\quad b = 2,\quad f(n) = \Theta(n)$$

$$n^{\log_b a} = n^{\log_2 4} = n^2,\quad f(n) = n = n^{\log_b a - 1},\quad \varepsilon = -1 < 0$$

符合主定理第一种情况，所以$T(n) = \Theta(n^2)$

算法复杂度：$\Theta(n^2)$

例3：位乘问题

③改进分治法：

$$XY = AC\ 2^n + (AD + BC)2^{n/2} + BD$$

代数变换：

$$AD + BC = (A - B)(D - C) + AC + BD$$

$$\begin{cases} W(n) = 3W(n/2) + cn \\ W(1) = 1 \end{cases}$$

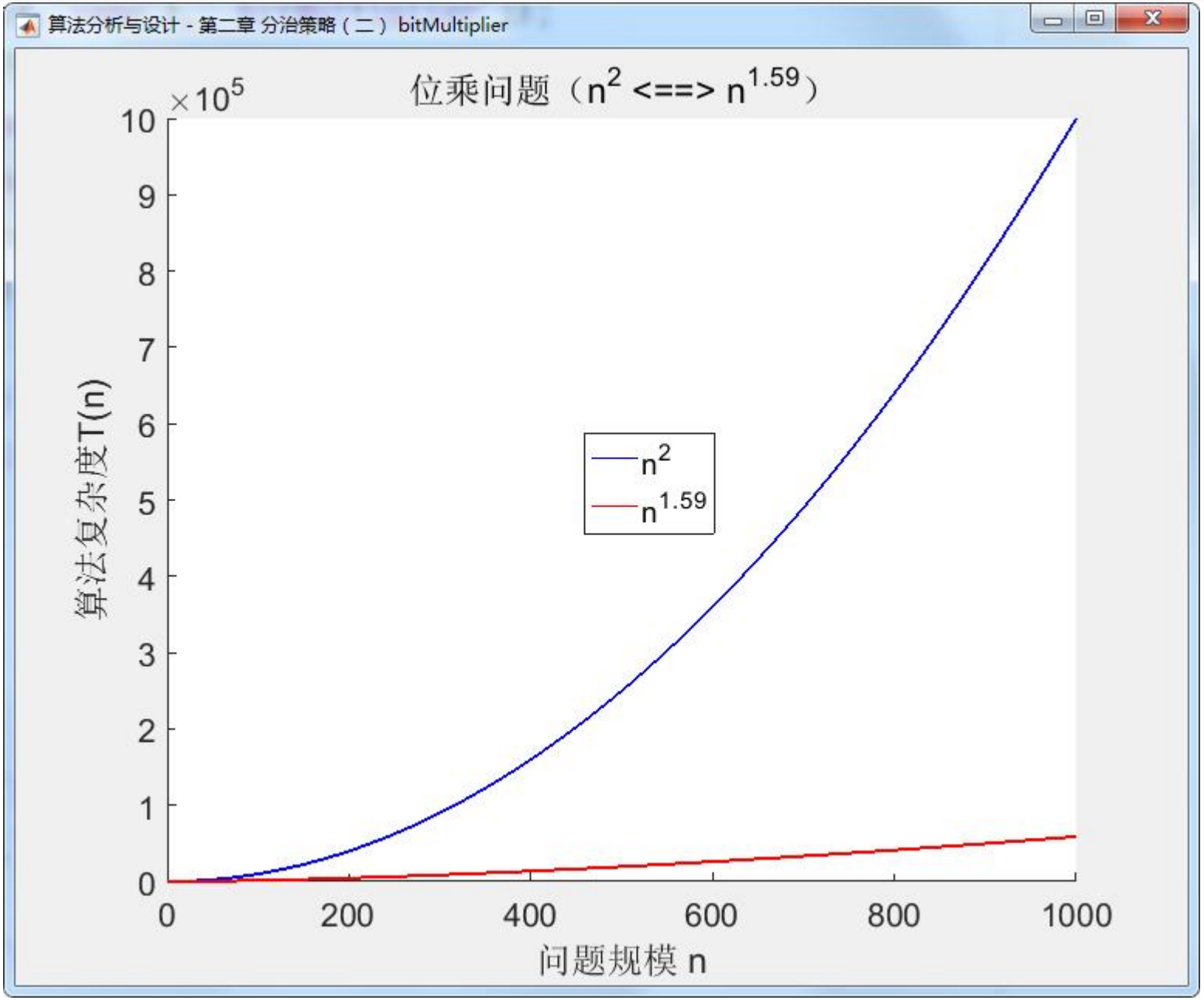
代入主定理：

$$W(n) = aW(n/b) + f(n),\quad a = 3,\quad b = 2,\quad f(n) = \Theta(n)$$

$$W(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.59})$$

算法复杂度： $\Theta(n^{1.59})$

算法分析：蛮力算法〈==〉分治算法



例4：矩阵乘法问题

例2.4 A,B 为两个n 阶矩阵，$n=2^k$,

计算 $C=AB$.

$$\begin{pmatrix} A_{11}A_{12}\ldots\ldots A_{1n} \\ A_{21}A_{22}\ldots\ldots A_{2n} \\ \ldots\ldots\ldots\ldots\ldots\ldots \\ A_{n1}A_{n2}\ldots\ldots A_{nn} \end{pmatrix}\begin{pmatrix} B_{11}B_{12}\ldots\ldots B_{1n} \\ B_{21}B_{22}\ldots\ldots B_{2n} \\ \ldots\ldots\ldots\ldots\ldots\ldots \\ B_{n1}B_{n2}\ldots\ldots B_{nn} \end{pmatrix}=\begin{pmatrix} C_{11}C_{12}\ldots\ldots C_{1n} \\ C_{21}C_{22}\ldots\ldots C_{2n} \\ \ldots\ldots\ldots\ldots\ldots\ldots \\ C_{n1}C_{n2}\ldots\ldots C_{nn} \end{pmatrix}$$

C矩阵其中第1列的元素：

$$C_{11}=A_{11}B_{11}+A_{12}B_{21}+\ldots\ldots+A_{1n}B_{n1}$$

$$C_{21}=A_{21}B_{12}+A_{22}B_{22}+\ldots\ldots+A_{2n}B_{n2}$$

$$\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots$$

$$C_{n1}=A_{n1}B_{1n}+A_{n2}B_{2n}+\ldots\ldots+A_{nn}B_{nn}$$

C矩阵第2列~第n列计算同上。

例4：矩阵乘法问题

蛮力(传统)算法： C = A×B 均是*n*阶矩阵

n阶矩阵 C = A*B

算法伪码如下：

```
for i=1 to n          //1到n列的元素
    for j=1 to n      //每列中1到n行的元素
        c[i][j] = 0
        for k=1 to n//每个元素为n项相乘之和
            c[i][j] += a[i][k] * b[k][j]
return C
```

例4：矩阵乘法问题

例2.4 A,B 为两个n 阶矩阵，$n=2^k$,

计算 $C=AB$.

①蛮力算法分析如下：

C中有n^2个元素

每个元素需要做n次乘法

以元素相乘为基本运算

算法复杂度：$W(n)=O(n^3)$

例4：矩阵乘法问题

②简单分治法：将矩阵分块，得

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

其中

$$C_{11} = A_{11}B_{11} + A_{12}B_{21} \qquad C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21} \qquad C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

递推方程：$W(n) = 8\,W(n/2) + cn^2$

$W(1) = 1$

解得：算法复杂度$W(n) = O(n^3)$

例4：矩阵乘法问题--Strassen 矩阵乘法

矩阵乘法是种极其耗时的运算，而在做科学运算时又是必不可少。

Volker Strassen，出生于1936年，德国数学家。在计算机科学和算法领域，因为Strassen算法而被人们所认识。Strassen算法是一种典型的分治算法，这个算法证明了复杂度为n^3的矩阵相乘算法不是最优解！他的工作进而触发了矩阵相乘领域更多的研究，从而产生了复杂度为$O(n^{2.376})$的Coppersmith-Winograd算法。

Coppersmith算法 https://www.cnblogs.com/lengyue365/p/5727144.html

程序清单 http://hongbomin.com/2016/07/19/fast-matrix-multiplication/

例4：矩阵乘法问题--Strassen 矩阵乘法

③Strassen矩阵乘法：

$M_1 = A_{11}(B_{12} - B_{22})$

$M_2 = (A_{11} + A_{12})B_{22}$

$M_3 = (A_{21} + A_{22})B_{11}$

$M_4 = A_{22}(B_{21} - B_{11})$

$M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$

$M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$

$M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$

$C_{11} = M_5 + M_4 - M_2 + M_6$

$C_{12} = M_1 + M_2$

$C_{21} = M_3 + M_4$

$C_{22} = M_5 + M_1 - M_3 - M_7$

时间复杂度：$O(n^{2.8075})$

$$W(n) = 7W(\tfrac{n}{2}) + 18(\tfrac{n}{2})^2$$

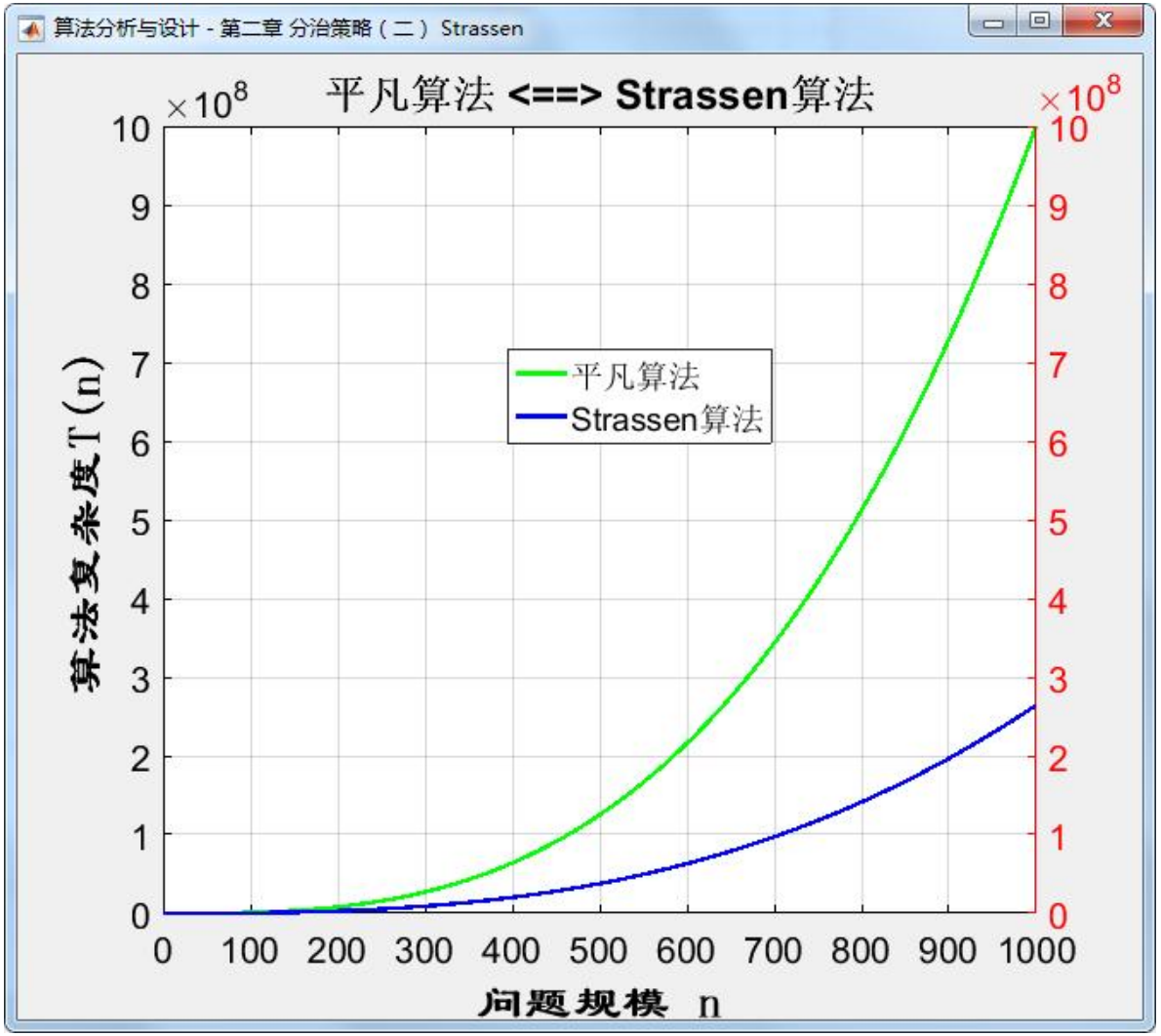
$$W(1) = 1$$

解得：

$$W(n) = O(n^{\log_2 7}) = O(n^{2.8075})$$

算法分析：蛮力算法<==>Strassen算法

蛮力算法和Strassen算法的性能差异，n越大，Strassen算法节约的时间越多。



2.4 典型实例分析

2.4.1 排序

(1) 快速排序

(2) 归并排序

2.4.2 选择问题

2.4.3 n-1次多项式在全体2n次根上求值

2.4.1 快速排序—从改进冒泡算法而来

冒泡排序算法：BubbleSort(A, n) 输入：数组$A[n]$

输出：排好序的数组A

伪代码：

```
for  i ← 1 to n-1 do
     for j ← 1 to n-i do
         if( A[j] > A[j+1] )A[j] ↔ A[j+1]
```

冒泡排序算法BubbleSort(A, n) //相临两个元素比较

i=1	27	99	0	8	13	64	86	16	7	10	88	25	90
首轮	j=1												n
i=2	27	0	8	13	64	86	16	7	10	88	25	90	99
二轮	j=1											n−1	
i=3	0	8	13	27	64	16	7	10	86	25	88	90	99
三轮	j=1										n−2		
i=4	0	8	13	27	16	7	10	64	25	86	88	90	99
四轮	j=1									n−3			
	……												
i=n−1	0	7	8	10	13	16	25	27	64	86	88	90	99
n−1轮	j=1	2											
结束	0	7	8	10	13	16	25	27	64	86	88	90	99

冒泡排序算法分析

共进行了n-1轮排序，

1轮进行了n-1次比较

2轮进行了n-2次比较

3轮进行了n-3次比较

……

n-1轮进行了1次比较

等差数列求和：$(n-1)*((n-1)+1)/2 = n^2/2 - n/2$

冒泡排序的算法复杂度：$W(n)=O(n^2)$

2.4.1 快速排序—分治算法

算法2.5 QuickSort(A, p, r)

输入：数组$A[p..r]$

输出：排好序的数组A

伪代码：

```
if p < r then
    q ← Partition(A, p, r)
    A[p] ↔ A[q]
    Quicksort(A, p, q-1)
    Quicksort(A, q+1, r)
```

初始置p=1，r=n，然后调用上述算法

2.4.1 快速排序算法——划分实例

算法2.6　Partition(A, p, r)　//查找第一个元素最终位置

首轮：	27	99	0	8	13	64	86	16	7	10	88	25	90
1次交换		i										j	
	27	25	0	8	13	64	86	16	7	10	88	99	90
2次交换						i				j			
	27	25	0	8	13	10	86	16	7	64	88	99	90
3次交换							i		j				
	27	25	0	8	13	10	7	16	86	64	88	99	90
4次交换								j	i				
	27	25	0	8	13	10	7	16	86	64	88	99	90

首轮结束：因为 $i < j$ 不成立，结束并返回 j

16	25	0	8	13	10	7	27	86	64	88	99	90

$A[p] \leftrightarrow A[j]$　　子问题1　　j　　子问题2

算法2.6 Partition(A, p, r) //查找第一个元素最终位置

输入：数组$A[p, r]$

输出：j，A的首元素在排好序的数组中的位置

伪代码：

```
x ← A[p]
i ← p          //先减后加，从第p+1个开始
j ← r+1        //先加后减，从第r个开始
while true do
    repeat  j ← j - 1
    until  A[ j ] ≤ x
    repeat  i ←  i +1
    until  A[ i ] > x
    if  i < j   then
        A[ i ] ↔ A[ j ]
        if i ==j-1 then return i
    else  return  j
```

2.4.1 快速排序算法——时间复杂度

快速排序算法复杂度分析：

最坏情况

$$W(n) = W(n-1) + n - 1$$

$$W(1) = 0$$

$$W(n) = \frac{1}{2}n(n-1) = \Theta(n^2)$$

最好划分

$$T(n) = 2T(\tfrac{n}{2}) + n - 1$$

$$T(1) = 0$$

$$T(n) = \Theta(n \log n)$$

均衡划分

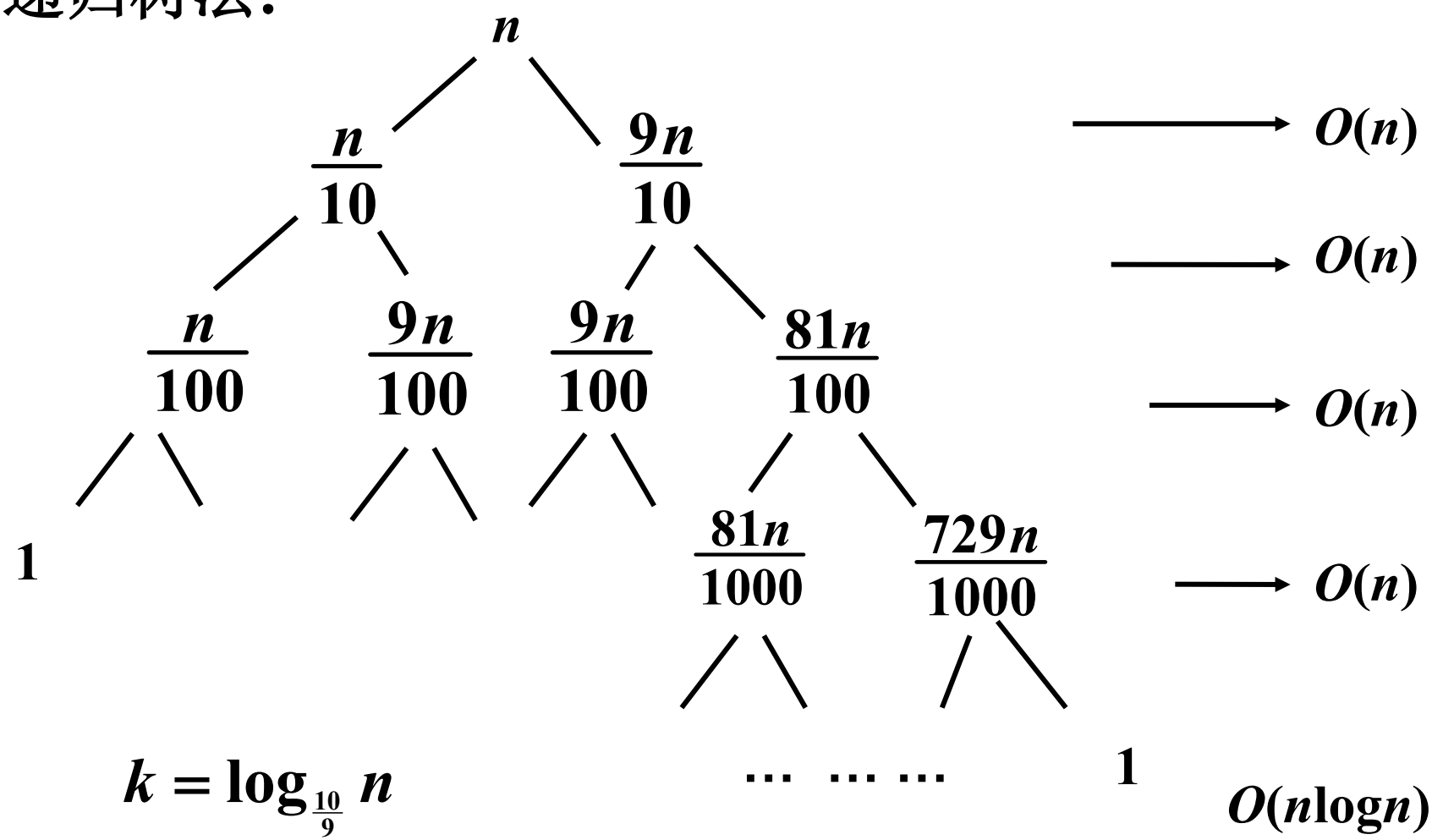
$$T(n) = T(\tfrac{9n}{10}) + T(\tfrac{n}{10}) + n$$

$$T(1) = 0$$

$$T(n) = \Theta(n \log n)$$

2.4.1 快速排序算法——均衡划分

递归树法：



2.4.1 快速排序算法——平均时间复杂度

假设输入数组首元素排好序后的正确位置在1，2，3，…，n各种情况四等可能的，概率为$1/n$。

$$\mathrm{T}(n)=\frac{1}{n}\{[T(0)+T(n-1)]+[T(1)+T(n-2)]+\cdots[T(n-1)+T(0)]\}+O(n)$$

$$=\frac{2}{n}\sum_{i=0}^{n-1}T(i)+O(n)\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots(T(0)=0)$$

$$=\frac{2}{n}\sum_{i=1}^{n-1}T(i)+O(n)$$

差消法解得：平均复杂度为$\Theta(n\log n)$

第一章③差消迭代 ： 差消化简后再迭代

$$\begin{cases} T(n) = \dfrac{2}{n}\sum_{i=1}^{n-1} T(i) + cn, \quad n \geq 1 \\ T(0) = 0 \end{cases}$$

快速排序平均时间分析

$$nT(n) = 2\sum_{i=1}^{n-1} T(i) + cn^2$$

$$(n-1)T(n-1) = 2\sum_{i=1}^{n-2} T(i) + c(n-1)^2$$

$$nT(n) = (n+1)T(n-1) + 2cn - c$$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2cn-c}{n(n+1)} = \cdots$$

$$= 2c[\frac{1}{n+1} + \frac{1}{n} + \ldots + \frac{1}{3} + \frac{T(1)}{2}] - O(\frac{1}{n}) = \Theta(\log n)$$

$$T(n) = \Theta(n \log n)$$

2.4.1 快速排序——程序代码

C++实现代码：

```
void swap(int& a, int& b){
    int temp = a;
    a = b;
    b = temp;
}
int partition(int A[], int p, int r){
    int x = A[p];
    int i = p;    //先减后加，从第p+1个开始
    int j = r+1;  //先加后减，从第r个开始
    while(true){
        while(A[++i] <= x);
        while(A[--j] > x);
        if( i<j ){
            swap(A[i], A[j]);
            if( i==j-1) return i;
        }
        else return j;
    }
}
```

2.4.1 快速排序——程序代码

C++实现代码：

```
#include <iostream>
using namespace std;
void quickSort(int A[], int p, int r){
    if( p < r ){
        int q = partition(A, p, r);
        swap(A[p], A[q]);
        quickSort(A, p, q-1);
        quickSort(A, q+1, r);
    }
}
int main(){
    int A[] = {17, 4, 98, 4, 33, 7, 8, 9, 3, 2, 8, 4, 27, 28, 7};
    int n = sizeof(A)/sizeof(int);
    quickSort(A, 0, n-1);
    for (int i=0; i<n; i++)cout << A[i] << ", ";
    cout << endl;
    return 0;
}
```

2.4 典型实例分析

2.4.1 排序

(1) 快速排序

(2) 归并排序

2.4.2 选择问题

2.4.3 n-1次多项式在全体2n次根上求值

二分归并排序 *binaryMergeSort*() 内容参见第2章第一讲

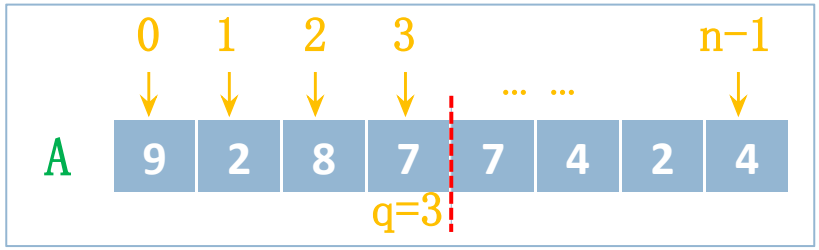
对有n个元素的数组A进行排序二分归并排序。

分析：

二分归并排序：如果n=1，算法结束。否则将数组A从中间分成左右两部分，分别对左部分和右部分进行二分归并排序，再将两部分有序数组合并成整体有序。

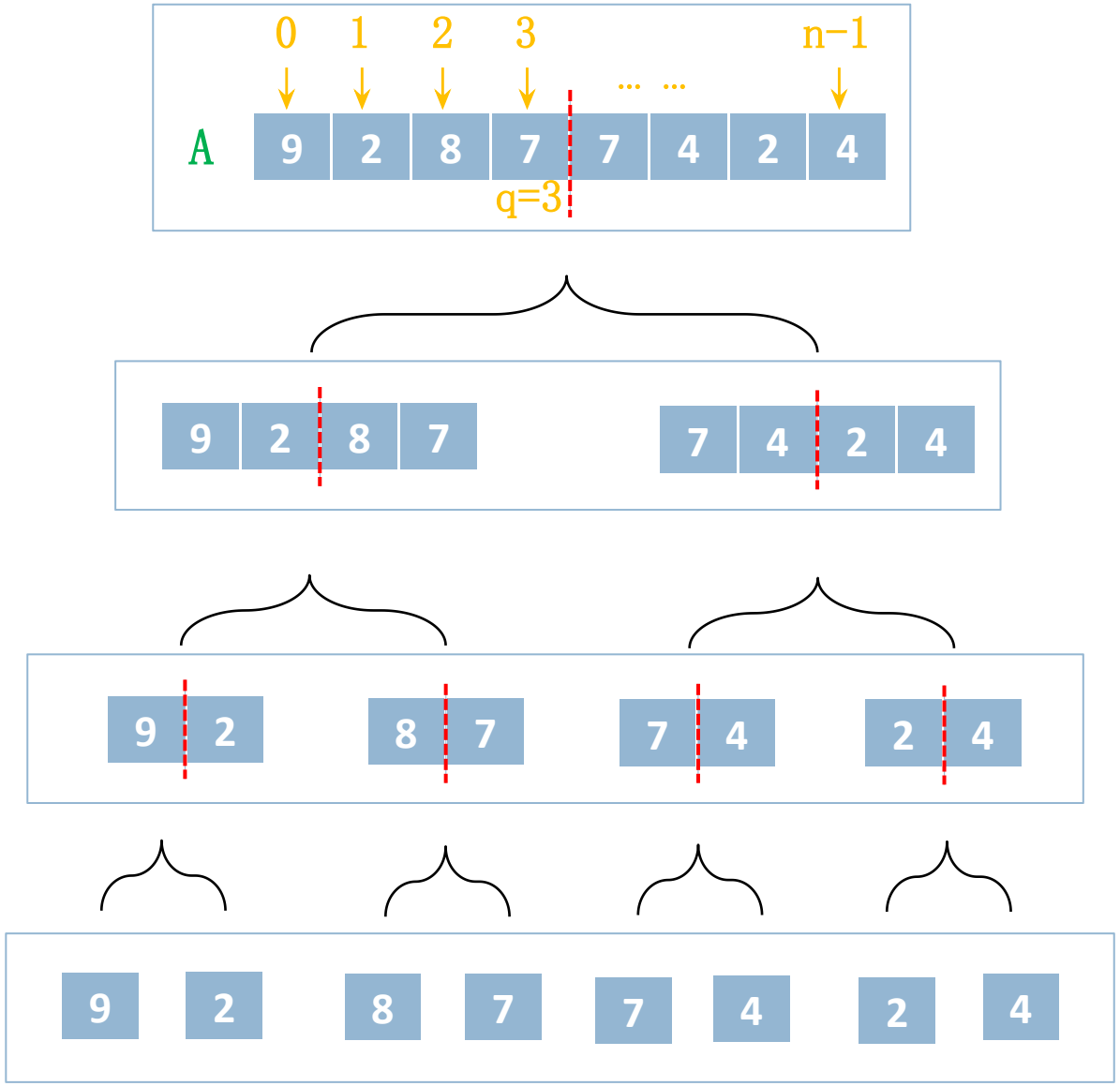
$O(n\log n)$ （空间复杂度$O(n)$）

l←0，r←n-1，q←(l+r)/2



2.4.2 归并排序——算法分析

$l \leftarrow 0$, $r \leftarrow n-1$, $q \leftarrow (l+r)/2$



(2) 归并排序（第1章换元迭代法）

二分归并排序数组 $A[p..r]$。将数组A中p到r的元素从中间分成左右两部分，分别把左右部分归并排序，再合并起来。

MergeSort(A,p,r)伪代码：

```
if  p<r  then
        q←(p+r)/2
        MergeSort(A,p,q)
        MergeSort(A,q+1,r)
        Merge(A,p,q,r)
```

算法1.6 Merge(A,p,q,r)//将排序数组$A[p..q]$与$A[q+1,r]$合并

```
x ← q - p+1, y ← r - q    // x, y分别为两个子数组的元素数
将A[p..q]复制到B[1..x]，将A[q+1..r]复制到C[1..y]
i← 1, j←1, k←p
while i ≤ x and j ≤ y do
    if B[i] ≤ C[j]  then   // B的首元素不大于C的首元素
        A[k]← B[i]      // 将B的首元素放到A中
        i ← i+1
    else
        A[k] ← C[j]
        j ← j+1
    k ← k+1
if i>x then 将C[j..y]复制到A[k..r]   // B已经是空数组
else 将B[i..x]复制到A[k..r]           // C已经是空数组
```

2.4.2 归并排序——复杂度分析

伪代码：

```
void binaryMergeSort(int A[], int left, int right)
    if left>=right then return
    q ← (left+right)/2
    binaryMergeSort(A, left, q)          T(n/2)
    binaryMergeSort(A, q+1, right)       T(n/2)
    merge(A, left, q, right)             O(n)

void merge(int A[], int left, int q, int right)
    复制A数组left~q到B数组，复制A数组q+1~right到C数组    O(n)
    i←0, j←0, k←left;
    while i<q-left+1 && j<right-q do
        if B[i]<=C[j] then A[k]←B[i], k←k+1, i←i+1      O(n)
        else A[k++]←C[j++], k←k+1, j←j+1
    }
    if i==q-left+1 then while j<right-q do A[k]=C[j++],k←k+1,j←j+1
    else while i<q-left+1 do A[k]=B[i], k←k+1, i←i+1
}
```

$$T(n)=2T(n/2)+O(n)$$

二分归并排序时间复杂度$O(n\log n)$，空间复杂度O(n)

算法分析

二分归并排序算法函数：

$$\begin{cases} W(n) = 2W(n/2) + n - 1, & n = 2^k \\ W(1) = 0 \end{cases}$$

主定理解法

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n, \quad f(n) = n - 1 = \Theta(n^{\log_b a + \varepsilon}), \quad \varepsilon = 0$$

主定理第二种情况解得 $T(n) = \Theta(n^1 \log n) = \Theta(n \log n)$

2.4.2 归并排序——程序代码

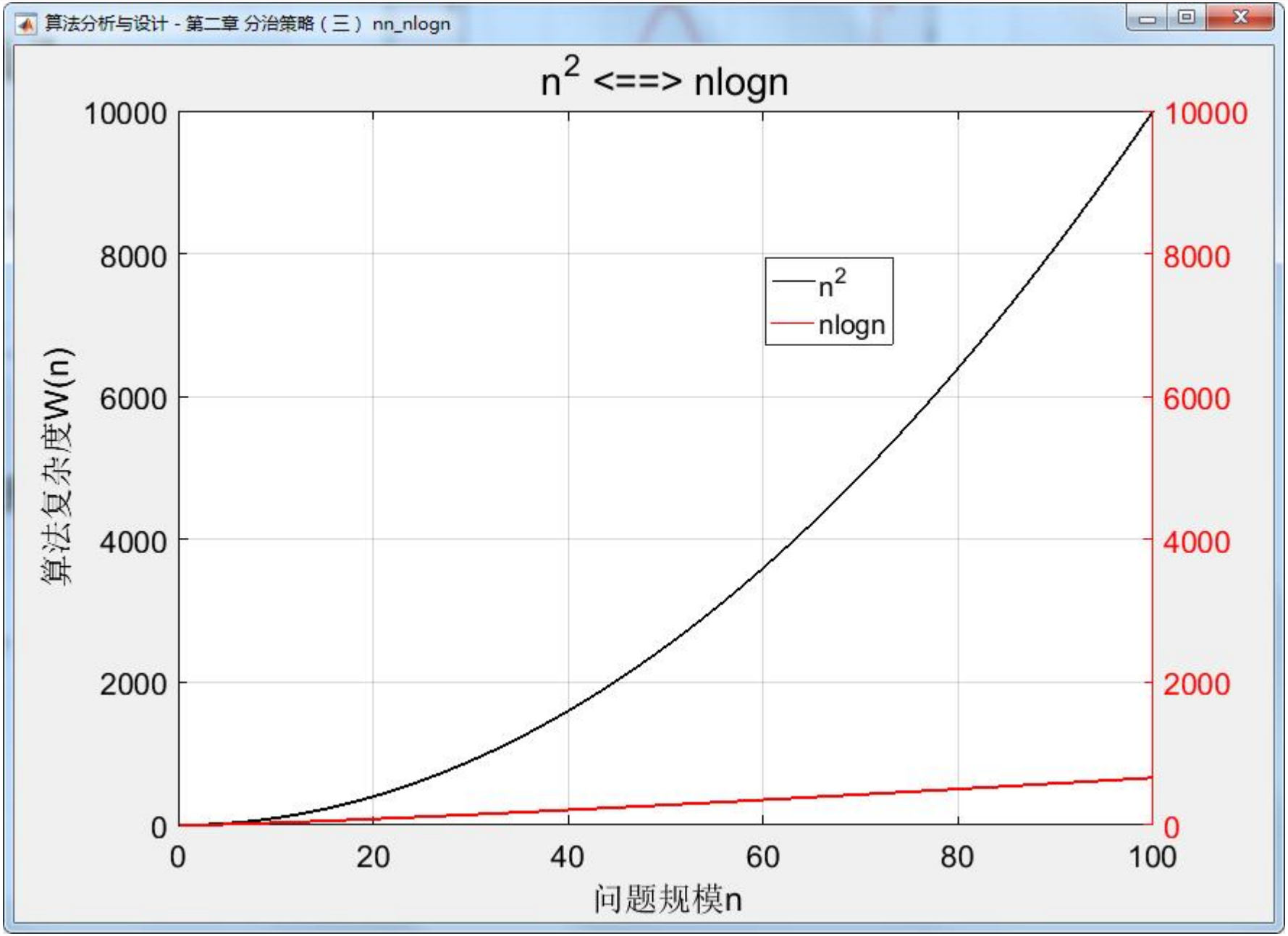
C++实现代码：

```
void merge(int A[], int left, int q, int right){
    int n = right-left+1;
    int B[q-left+1], C[right-q];
    for(int i=0; i<=q-left; i++)B[i]=A[left+i];
    for(int i=0; i<=right-q-1; i++)C[i]=A[q+1+i];
    int i=0, j=0, k=left;
    while(i<q-left+1 && j<right-q){
        if(B[i]<=C[j])A[k++]=B[i++];
        else A[k++]=C[j++];
    }
    if(i==q-left+1)while(j<right-q)A[k++]=C[j++];
    else while(i<q-left+1)A[k++]=B[i++];
}
```

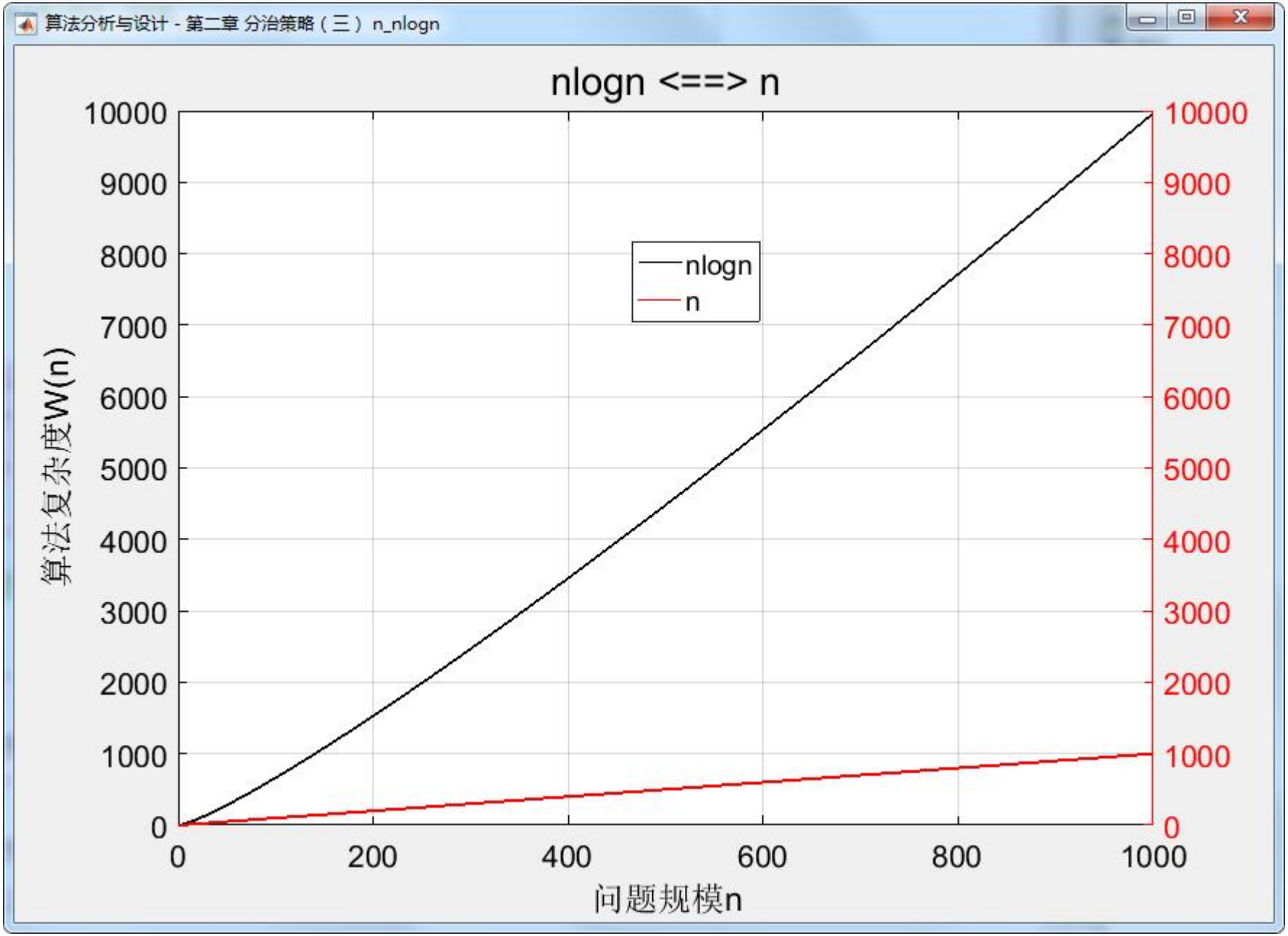
C++实现代码：

```
#include <iostream>
using namespace std;

void binaryMergeSort(int A[], int left, int right){
    if(left>=right)return;
    int q = (left+right)/2;
    binaryMergeSort(A, left, q);
    binaryMergeSort(A, q+1, right);
    merge(A, left, q, right);
}

int main(){
    int A[] = {17, 4, 98, 4, 33, 7, 8, 9, 3, 2, 8, 4, 27, 28, 7};
    int n = sizeof(A)/sizeof(int);
    binaryMergeSort(A, 0, n-1);
    for (int i=0; i<n; i++)cout << A[i] << ", ";
    cout << endl;
    return 0;
}
```

2.4.1 排序算法——复杂度对比



2.4.1 排序算法——复杂度对比



2.4.2 归并排序——算法分析

伪代码： 参见第2章第一讲，此处的伪代码更接近C++语言

```
binaryMergeSort(int A[], int left, int right)
    if left>=right then return
    q ← (left+right)/2
    binaryMergeSort(A, left, q)
    binaryMergeSort(A, q+1, right)
    merge(A, left, q, right)

void merge(int A[], int left, int q, int right)
    复制A数组left~q到B数组，复制A数组q+1~right到C数组
    i←0, j←0, k←left;
    while i<q-left+1 && j<right-q do
        if B[i]<=C[j] then A[k]←B[i], k←k+1, i←i+1
        else A[k++]←C[j++], k←k+1, j←j+1
    }
    if i==q-left+1 then while j<right-q do A[k]=C[j++],k←k+1,j←j+1
    else while i<q-left+1 do A[k]=B[i], k←k+1, i←i+1
}
```

2.4.1 排序算法——小结

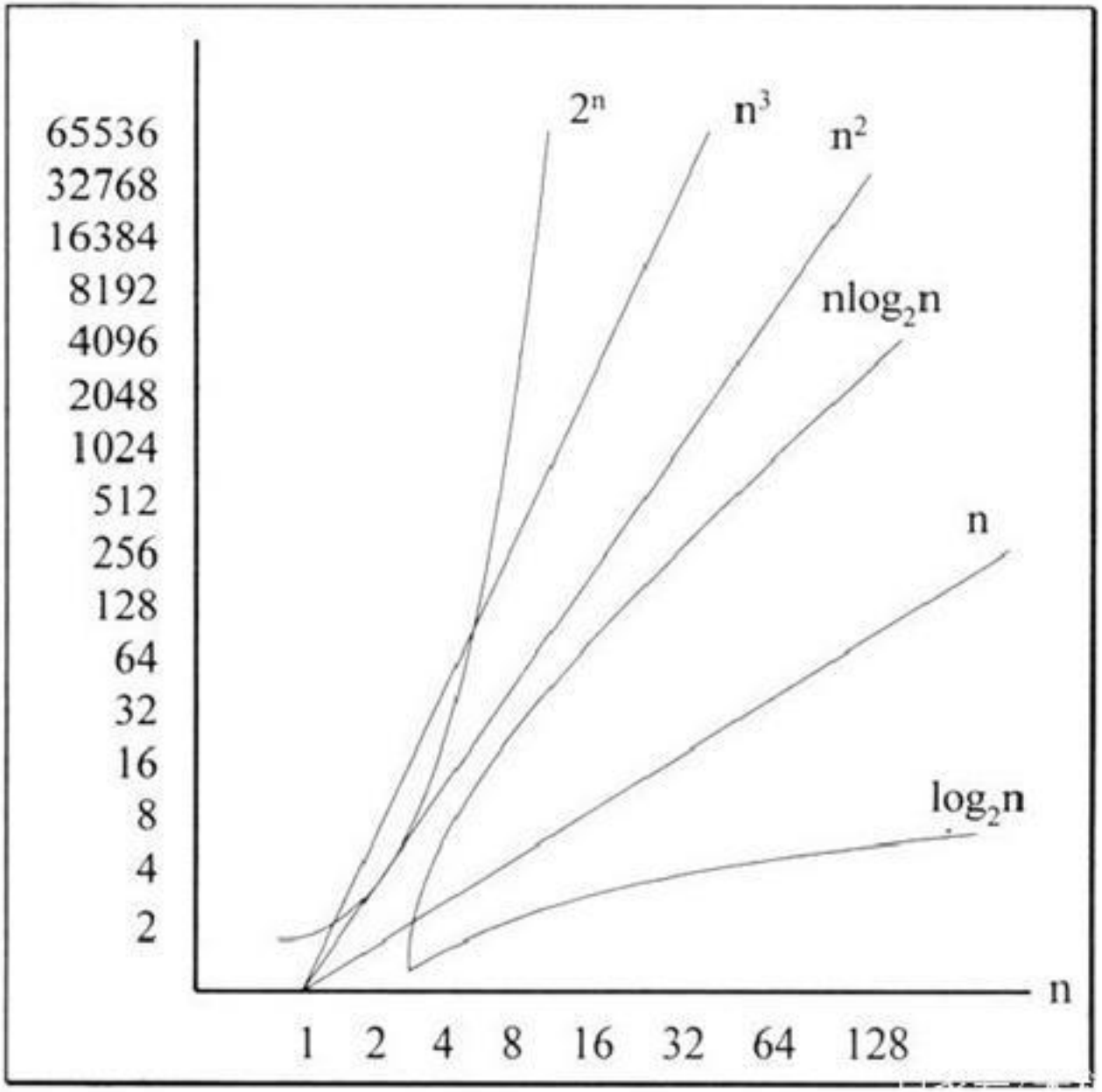
十大排序算法

1. 插入排序
2. 选择排序
3. 冒泡排序
4. 希尔排序
5. 快速排序
6. 归并排序
7. 堆排序
8. 计数排序
9. 桶排序
10. 基数排序

2.4.1 排序算法——小结

	排序方法	时间复杂度(最坏)	时间复杂度(平均)	空间复杂度	稳定性
比较排序	插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
	冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
	选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
	希尔排序	$O(n^2)$	$O(n^{1.3})$	$O(1)$	不稳定
	快速排序	$O(n^2)$	$O(n\log n)$	$O(\log n)\sim O(n)$	不稳定
	堆排序	$O(n\log n)$	$O(n\log n)$	$O(1)$	不稳定
	归并排序	$O(n\log n)$	$O(n\log n)$	$O(n)$	稳定
非比较排序	基数排序	$O(n*k)$	$O(n*k)$	$O(n+k)$	稳定
	桶排序	$O(n^2)$	$O(n+k)$	$O(n+k)$	稳定
	计数排序	$O(n+k)$	$O(n+k)$	$O(n+k)$	稳定

2.4.1 排序算法——小结


65536
32768
16384
8192
4096
2048
1024
512
256
128
64
32
16
8
4
2
2^n
n^3
n^2
$n\log_2 n$
n
$\log_2 n$
n
1 2 4 8 16 32 64 128

2.4.1 排序算法--小结

排序的稳定性

如果两个相等元素在排序前后位置不变，则排序是稳定的。

如果排序有可能引起两个相等元素在排序前后位置变化，则排序是不稳定的。

本节课小结

对以上问题分析：

根据 $W(n) = aW(n/b)+f(n)$

提高效率的方法：

1. 减少子问题的个数a

$$W(n) = O(n^{\log_b a})$$

2. 增加预处理，减少$f(n)$规模

本节课小结

改进分治算法的途径：

①减少问题的个数。

②降低子问题的规模。

③把子问题内的工作量移到子问题外。

④降低子问题分解合并方面的工作量。

⑤增加预处理，减少$f(n)$规模

作 业

作业四：第二章习题2

P48 2.7、2.8、二分归并排序、快速排序

1. 写出算法伪代码

2. 分析算法复杂度

结 束

谢谢收看，再见！